



Deep Learning

Tutorial 14: Building a Simple RNN for Named Entity Recognition (NER)

Submitted By: **Muhammad Mubashar**

Submitted To: **Dr Shahbaz Khan**

Reg No # **000000494171**

Semester: **4th**

Department of Robotics & Artificial Intelligence

School of Mechanical & Manufacturing Engineering

NUST

Tasks:

Make your own data set and test it with the model
Change the number of units, epochs & learning rate

Implementation of these tasks are in **PyTorch**:

```
# =====
```

```
# Tutorial 14 – Simple RNN for Named Entity Recognition (NER)
```

```
# Implemented in PyTorch | Ready for Google Colab
```

```
# =====
```

```
# Objectives:
```

```
# • Understand the architecture of RNN
```

```
# • How to implement the RNN in PyTorch
```

```
# • Visualize the results
```

```
#
```

```
# TASKS COVERED:
```

```
# ✓ Original dataset (Barack Obama / Google sentences)
```

```
# ✓ Custom dataset (tech-company / scientist sentences)
```

```
# ✓ Experiment with units, epochs & learning rate
```

```
# =====
```

```
# — 0. Install / import —————
```

```
# (All packages below are pre-installed in Google Colab)
```

```
import torch
```

```
import torch.nn as nn
```

```
import torch.optim as optim
```

```
from torch.utils.data import Dataset, DataLoader
```

```
from collections import defaultdict
```

```

import numpy as np

import matplotlib.pyplot as plt

import matplotlib.patches as mpatches

from sklearn.preprocessing import LabelEncoder


# Reproducibility

torch.manual_seed(42)

np.random.seed(42)


device = "cuda" if torch.cuda.is_available() else "cpu"

print(f"Using device: {device}")

```

```

# =====

# SECTION 1 – ORIGINAL DATASET (from Tutorial)

# =====

print("\n" + "="*60)

print("SECTION 1 – Original Dataset")

print("="*60)

```

```

# — 1.1 Data —————

sentences = [

    ["Barack", "Obama", "was", "born", "in", "Hawaii"],

    ["Google", "is", "based", "in", "Mountain", "View"],

```

```
]
```

```
labels = [
```

```
    ["PERSON",    "PERSON",    "O", "O", "O", "LOCATION"],
```

```
    ["ORGANIZATION", "O",        "O", "O", "LOCATION", "O"],
```

```
]
```

```
# — 1.2 Vocabulary & label encoder —————
```

```
PAD_TOKEN = "<PAD>"
```

```
UNK_TOKEN = "<UNK>"
```

```
all_words = [w.lower() for sent in sentences for w in sent]
```

```
all_labels = [l for lab in labels for l in lab]
```

```
word2idx = {PAD_TOKEN: 0, UNK_TOKEN: 1}
```

```
for w in all_words:
```

```
    if w not in word2idx:
```

```
        word2idx[w] = len(word2idx)
```

```
label_encoder = LabelEncoder()
```

```
label_encoder.fit(["O", "PERSON", "LOCATION", "ORGANIZATION"])
```

```
num_classes = len(label_encoder.classes_)
```

```
print(f"Label classes : {list(label_encoder.classes_)}")
```

```
print(f'Vocabulary size: {len(word2idx)}')
```

```
# — 1.3 Encode & pad —————
```

```
def encode_sentence(sent):
```

```
    return [word2idx.get(w.lower(), word2idx[UNK_TOKEN]) for w in sent]
```

```
def encode_labels(lab):
```

```
    return label_encoder.transform(lab).tolist()
```

```
max_len = max(len(s) for s in sentences)
```

```
def pad_sequence_manual(seq, max_len, pad_val=0):
```

```
    return seq + [pad_val] * (max_len - len(seq))
```

```
X = [pad_sequence_manual(encode_sentence(s), max_len) for s in sentences]
```

```
y = [pad_sequence_manual(encode_labels(l), max_len, pad_val=0) for l in labels]
```

```
X_tensor = torch.tensor(X, dtype=torch.long).to(device)
```

```
y_tensor = torch.tensor(y, dtype=torch.long).to(device)
```

```
print(f'X shape: {X_tensor.shape}, y shape: {y_tensor.shape}')
```

```
# — 1.4 RNN Model —————
```

```
class SimpleRNNTagger(nn.Module):
```

```
"""
```

Embedding → SimpleRNN → Dropout → TimeDistributed Linear (softmax)

```
"""
```

```
def __init__(self, vocab_size, embed_dim, hidden_units, num_classes, dropout=0.1):
```

```
    super().__init__()
```

```
    self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)
```

```
    self.rnn = nn.RNN(embed_dim, hidden_units,  
                      batch_first=True, nonlinearity='tanh')
```

```
    self.dropout = nn.Dropout(dropout)
```

```
    self.fc = nn.Linear(hidden_units, num_classes) # TimeDistributed
```

```
def forward(self, x):
```

```
    emb = self.embedding(x)      # (B, T, E)
```

```
    out, _ = self.rnn(emb)       # (B, T, H)
```

```
    out = self.dropout(out)
```

```
    logits = self.fc(out)       # (B, T, C)
```

```
    return logits
```

— 1.5 Training function —————

```
def train_model(model, X, y, epochs, lr, label=""):
```

```
    optimizer = optim.Adam(model.parameters(), lr=lr)
```

```
    criterion = nn.CrossEntropyLoss()
```

```
    loss_history = []
```

```

for epoch in range(epochs):

    model.train()

    optimizer.zero_grad()

    logits = model(X)                # (B, T, C)

    B, T, C = logits.shape

    loss = criterion(logits.view(B*T, C), y.view(B*T))

    loss.backward()

    optimizer.step()

    loss_history.append(loss.item())

    if (epoch + 1) % max(1, epochs // 5) == 0:

        print(f" [{label}] Epoch {epoch+1}/{epochs} Loss: {loss.item():.4f}")

return loss_history

```

— 1.6 Hyperparameter experiments —————

```

configs = [

    {"hidden_units": 50, "epochs": 30, "lr": 0.01, "label": "units=50 | epochs=30 | lr=0.01"},

    {"hidden_units": 100, "epochs": 50, "lr": 0.005, "label": "units=100 | epochs=50 | lr=0.005"},

    {"hidden_units": 30, "epochs": 20, "lr": 0.05, "label": "units=30 | epochs=20 | lr=0.05"},

]

all_histories = {}

```

for cfg in configs:

```
print(f"\nTraining: {cfg['label']}")
```

```
model = SimpleRNNTagger(
```

```
    vocab_size = len(word2idx),
```

```
    embed_dim = 50,
```

```
    hidden_units = cfg["hidden_units"],
```

```
    num_classes = num_classes,
```

```
    dropout = 0.1
```

```
).to(device)
```

```
hist = train_model(model, X_tensor, y_tensor,
```

```
                  cfg["epochs"], cfg["lr"], cfg["label"])
```

```
all_histories[cfg["label"]] = hist
```

— 1.7 Plot training curves —————

```
plt.figure(figsize=(10, 4))
```

```
for label, hist in all_histories.items():
```

```
    plt.plot(hist, label=label)
```

```
plt.title("Tutorial 14 – RNN Training Loss (Hyperparameter Comparison)")
```

```
plt.xlabel("Epoch")
```

```
plt.ylabel("Cross-Entropy Loss")
```

```
plt.legend(fontsize=8)
```

```
plt.tight_layout()
```

```
plt.savefig("Tutorial14_loss_curves.png", dpi=150)
```

```

plt.show()

print("Saved: Tutorial14_loss_curves.png")

# — 1.8 Test with a new sentence (Tutorial requirement) —————

# Use the best model (second config)

best_model = SimpleRNNTagger(len(word2idx), 50, 100, num_classes, 0.1).to(device)

_ = train_model(best_model, X_tensor, y_tensor, 80, 0.005, "best-model")

def predict_ner(model, sentence):

    model.eval()

    tokens = sentence.split()

    idxs = [word2idx.get(t.lower(), word2idx[UNK_TOKEN]) for t in tokens]

    # pad to max_len if needed

    padded = pad_sequence_manual(idxs, max(max_len, len(idxs)))

    inp = torch.tensor([padded], dtype=torch.long).to(device)

    with torch.no_grad():

        logits = model(inp)          # (1, T, C)

    preds = logits.argmax(dim=-1)[0].cpu().numpy()

    decoded = label_encoder.inverse_transform(preds[:len(tokens)])

    return list(zip(tokens, decoded))

test_sentence = "Barack Obama went to Hawaii"

results = predict_ner(best_model, test_sentence)

```

```
print("\n—— NER Predictions (Original Dataset) ——")
```

```
for word, label in results:
```

```
    print(f" Word: {word:15s} → Predicted Label: {label}")
```

```
# —— 1.9 Visualize NER predictions ——
```

```
label_colors = {
```

```
    "PERSON":    "#FF6B6B",
```

```
    "LOCATION":   "#4ECDC4",
```

```
    "ORGANIZATION": "#45B7D1",
```

```
    "O":        "#D3D3D3",
```

```
}
```

```
fig, ax = plt.subplots(figsize=(10, 2))
```

```
ax.axis("off")
```

```
x_pos = 0.02
```

```
for word, lbl in results:
```

```
    color = label_colors.get(lbl, "#CCCCCC")
```

```
    ax.text(x_pos, 0.5, word, fontsize=14, ha="left", va="center",
```

```
           bbox=dict(facecolor=color, edgecolor="black", boxstyle="round,pad=0.4"),
```

```
           transform=ax.transAxes)
```

```
    ax.text(x_pos, 0.15, lbl, fontsize=8, ha="left", va="center",
```

```
           color="gray", transform=ax.transAxes)
```

```
    x_pos += 0.17
```

```
patches = [mpatches.Patch(color=c, label=l) for l, c in label_colors.items()]  
ax.legend(handles=patches, loc="upper right", fontsize=8,  
          bbox_to_anchor=(1.0, 1.3), framealpha=0.8)  
plt.title("NER Predictions – Original Sentence", pad=20)  
plt.tight_layout()  
plt.savefig("Tutorial14_ner_predictions.png", dpi=150, bbox_inches="tight")  
plt.show()  
print("Saved: Tutorial14_ner_predictions.png")
```

```

=====
SECTION 1 - Original Dataset
=====
Label classes : [np.str_('LOCATION'), np.str_('O'), np.str_('ORGANIZATION'), np.str_('PERSON')]
Vocabulary size: 13
X shape: torch.Size([2, 6]), y shape: torch.Size([2, 6])

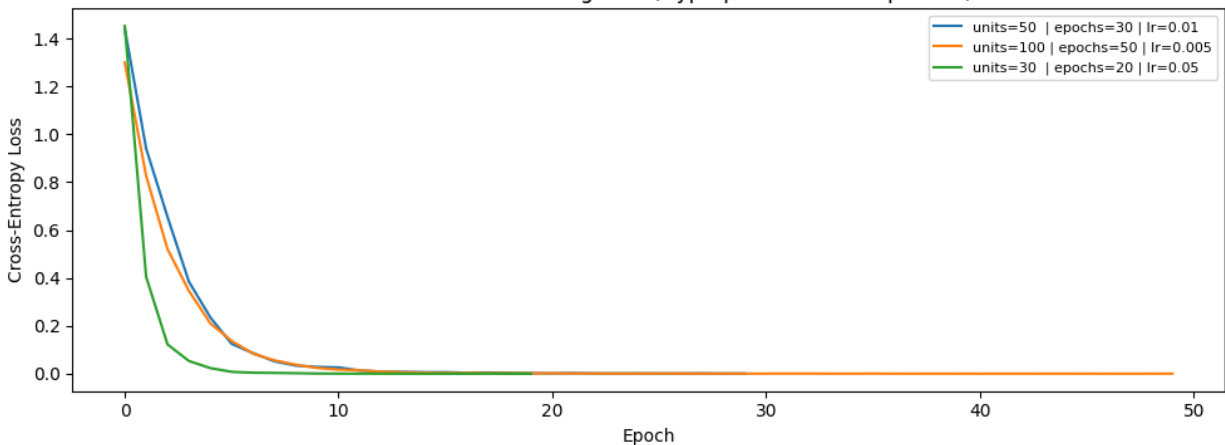
Training: units=50 | epochs=30 | lr=0.01
[units=50 | epochs=30 | lr=0.01] Epoch 6/30 Loss: 0.1248
[units=50 | epochs=30 | lr=0.01] Epoch 12/30 Loss: 0.0136
[units=50 | epochs=30 | lr=0.01] Epoch 18/30 Loss: 0.0052
[units=50 | epochs=30 | lr=0.01] Epoch 24/30 Loss: 0.0014
[units=50 | epochs=30 | lr=0.01] Epoch 30/30 Loss: 0.0010

Training: units=100 | epochs=50 | lr=0.005
[units=100 | epochs=50 | lr=0.005] Epoch 10/50 Loss: 0.0242
[units=100 | epochs=50 | lr=0.005] Epoch 20/50 Loss: 0.0018
[units=100 | epochs=50 | lr=0.005] Epoch 30/50 Loss: 0.0006
[units=100 | epochs=50 | lr=0.005] Epoch 40/50 Loss: 0.0005
[units=100 | epochs=50 | lr=0.005] Epoch 50/50 Loss: 0.0004

Training: units=30 | epochs=20 | lr=0.05
[units=30 | epochs=20 | lr=0.05] Epoch 4/20 Loss: 0.0534
[units=30 | epochs=20 | lr=0.05] Epoch 8/20 Loss: 0.0035
[units=30 | epochs=20 | lr=0.05] Epoch 12/20 Loss: 0.0005
[units=30 | epochs=20 | lr=0.05] Epoch 16/20 Loss: 0.0002
[units=30 | epochs=20 | lr=0.05] Epoch 20/20 Loss: 0.0002

```

Tutorial 14 - RNN Training Loss (Hyperparameter Comparison)



```

Saved: Tutorial14_loss_curves.png
[best-model] Epoch 16/80 Loss: 0.0049
[best-model] Epoch 32/80 Loss: 0.0008
[best-model] Epoch 48/80 Loss: 0.0004
[best-model] Epoch 64/80 Loss: 0.0004
[best-model] Epoch 80/80 Loss: 0.0003

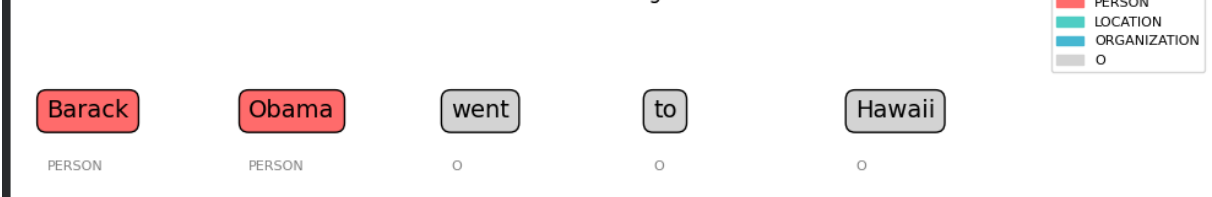
```

```

--- NER Predictions (Original Dataset) ---
Word: Barack      -> Predicted Label: PERSON
Word: Obama       -> Predicted Label: PERSON
Word: went        -> Predicted Label: O
Word: to          -> Predicted Label: O
Word: Hawaii      -> Predicted Label: O

```

NER Predictions - Original Sentence



```

Saved: Tutorial14_ner_predictions.png

```

```
# =====

# SECTION 2 – CUSTOM DATASET (TASK: Make your own dataset)

# =====

print("\n" + "="*60)

print("SECTION 2 – Custom Dataset (Task Solution)")

print("="*60)


custom_sentences = [

    ["Elon", "Musk", "founded", "SpaceX", "in", "California"],

    ["Albert", "Einstein", "developed", "relativity", "in", "Germany"],

    ["Apple", "is", "headquartered", "in", "Cupertino", "USA"],

    ["Marie", "Curie", "worked", "in", "Paris", "France"],

    ["Microsoft", "launched", "Windows", "from", "Seattle", "USA"],

]


custom_labels = [

    ["PERSON", "PERSON", "O", "ORGANIZATION", "O", "LOCATION"],

    ["PERSON", "PERSON", "O", "O", "O", "LOCATION"],

    ["ORGANIZATION", "O", "O", "O", "LOCATION", "LOCATION"],

    ["PERSON", "PERSON", "O", "O", "LOCATION", "LOCATION"],

    ["ORGANIZATION", "O", "O", "O", "LOCATION", "LOCATION"],

]
```

```

# Rebuild vocabulary including custom words

custom_all_words = [w.lower() for sent in custom_sentences for w in sent]

custom_word2idx = {PAD_TOKEN: 0, UNK_TOKEN: 1}

for w in custom_all_words:

    if w not in custom_word2idx:

        custom_word2idx[w] = len(custom_word2idx)


custom_max_len = max(len(s) for s in custom_sentences)

Xc = [pad_sequence_manual(

    [custom_word2idx.get(w.lower(), 1) for w in s], custom_max_len)

    for s in custom_sentences]

yc = [pad_sequence_manual(encode_labels(l), custom_max_len) for l in custom_labels]


Xc_tensor = torch.tensor(Xc, dtype=torch.long).to(device)

yc_tensor = torch.tensor(yc, dtype=torch.long).to(device)


custom_model = SimpleRNNTagger(

    vocab_size = len(custom_word2idx),

    embed_dim = 50,

    hidden_units = 100,

    num_classes = num_classes,

    dropout = 0.1

```

```

).to(device)

print("\nTraining on custom dataset ...")

custom_hist = train_model(custom_model, Xc_tensor, yc_tensor,
                           epochs=100, lr=0.005, label="custom-dataset")

# Test on a new custom sentence

def predict_ner_custom(model, sentence, word2idx, max_len):
    model.eval()

    tokens = sentence.split()

    idxs = [word2idx.get(t.lower(), word2idx[UNK_TOKEN]) for t in tokens]

    padded = pad_sequence_manual(idxs, max(max_len, len(idxs)))

    inp = torch.tensor([padded], dtype=torch.long).to(device)

    with torch.no_grad():
        logits = model(inp)

    preds = logits.argmax(dim=-1)[0].cpu().numpy()

    decoded = label_encoder.inverse_transform(preds[:len(tokens)])

    return list(zip(tokens, decoded))

custom_test = "Elon Musk visited France and Germany"

custom_results = predict_ner_custom(custom_model, custom_test,
                                     custom_word2idx, custom_max_len)

print("\n—— NER Predictions (Custom Dataset) ——")

```

```
for word, lbl in custom_results:

    print(f" Word: {word:15s} → Predicted Label: {lbl}")


# Loss curve for custom dataset

plt.figure(figsize=(8, 4))

plt.plot(custom_hist, color="#E74C3C")

plt.title("Tutorial 14 – Custom Dataset Training Loss")

plt.xlabel("Epoch"); plt.ylabel("Loss")

plt.tight_layout()

plt.savefig("Tutorial14_custom_loss.png", dpi=150)

plt.show()

print("Saved: Tutorial14_custom_loss.png")

print("\n✅ Tutorial 14 Complete!")
```

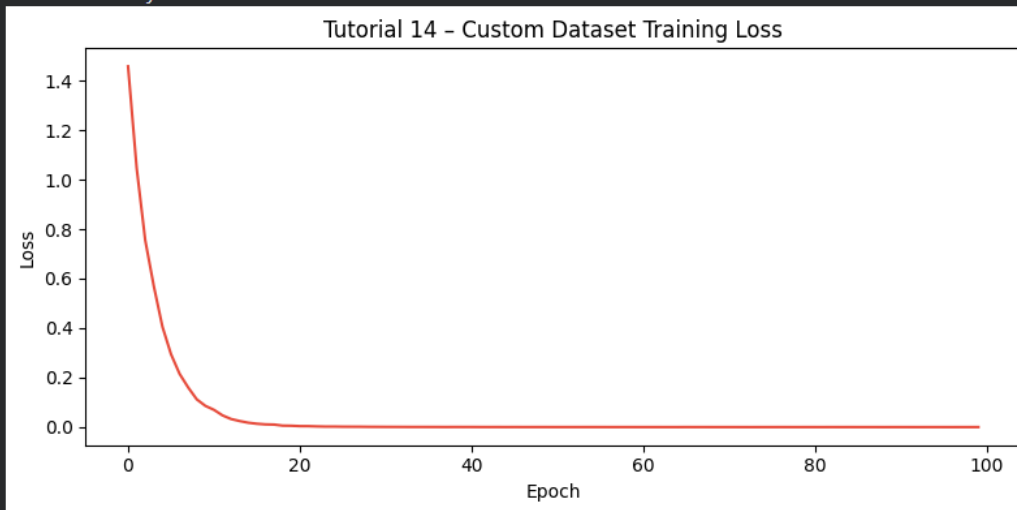
```
=====
SECTION 2 - Custom Dataset (Task Solution)
=====
```

```
Training on custom dataset ...
```

```
[custom-dataset] Epoch 20/100 Loss: 0.0057
[custom-dataset] Epoch 40/100 Loss: 0.0007
[custom-dataset] Epoch 60/100 Loss: 0.0004
[custom-dataset] Epoch 80/100 Loss: 0.0004
[custom-dataset] Epoch 100/100 Loss: 0.0004
```

```
— NER Predictions (Custom Dataset) —
```

```
Word: Elon      → Predicted Label: PERSON
Word: Musk      → Predicted Label: PERSON
Word: visited   → Predicted Label: O
Word: France    → Predicted Label: ORGANIZATION
Word: and       → Predicted Label: LOCATION
Word: Germany   → Predicted Label: LOCATION
```



```
Saved: Tutorial14_custom_loss.png
```

```
✅ Tutorial 14 Complete!
```